Group Topic: Predicting Student Dropout Risk Using Academic and Behavioral Data
Git Repo/Colab Link:
https://colab.research.google.com/drive/1whQOU1AtUXaLGr6YHy3VUAmJ5weENodt?usp=sharing

Role: Pipeline Lead (Data & Preprocessing)
Lab No.: 4
Date: May 16, 2026

Objectives

The purpose of this lab is to develop an end-to-end machine learning preprocessing pipeline for predicting student dropout risk using academic and behavioral data from the OULAD dataset. My personal objective as the **Pipeline Lead (Data & Preprocessing)** was to prepare the dataset for machine learning by implementing feature selection, categorical encoding, missing value handling, feature scaling, and train-test splitting.

This preprocessing pipeline ensures that the dataset is clean, standardized, and ready for classification models such as Logistic Regression, Random Forest, and XGBoost. Proper preprocessing is important because machine learning models rely on consistent numerical input and balanced training procedures to achieve reliable results. The completed pipeline also supports professional visualizations and evaluation metrics for model comparison.

Tasks Completed

The following preprocessing and pipeline integration tasks were completed to prepare the dataset for model training and evaluation.

1. Imported Required Libraries

I imported the necessary libraries for data preprocessing, visualization, machine learning, and evaluation metrics.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
```

2. Loaded the Cleaned Dataset

I loaded the cleaned master dataset containing academic and behavioral information from the OULAD dataset.

Code:

```
df = pd.read_csv('/content/oulad_cleaned_master.csv')
```

This dataset contains student-related variables used to predict dropout status.

---

3. Performed Feature Selection

I removed unnecessary columns that should not be used during training, including identifiers and target leakage variables.

Code:

```
exclude_cols = ['id_student', 'code_module', 'code_presentation',
                'final_result', 'risk_level', 'is_dropout']

feature_cols = [col for col in df.columns if col not in exclude_cols]

X = df[feature_cols].copy()
y = df['is_dropout'].copy()
```

This ensured that only meaningful predictive features were included in the machine learning pipeline.

---

4. Encoded Categorical Features

Since machine learning models require numerical input, I transformed categorical columns using Label Encoding.

Code:

```
categorical_cols = X.select_dtypes(include=['object']).columns

label_encoders = {}

for col in categorical_cols:
    le = LabelEncoder()
    X.loc[:, col] = le.fit_transform(X[col].astype(str))
    label_encoders[col] = le
```

This converted text-based categories into numeric values while preserving class distinctions.

---

5. Handled Missing Values

I implemented missing value imputation to ensure the dataset contained no null values before training.

Code:

```
for col in X.columns:
    if X[col].dtype == 'object':
        X.loc[:, col] = X[col].fillna(X[col].mode()[0])
    else:
        X.loc[:, col] = X[col].fillna(X[col].median())
```

Categorical columns were filled using the mode, while numerical columns used the median to reduce the effect of outliers.

---

6. Standardized Numerical Features

I scaled the dataset using StandardScaler to normalize feature values and improve model performance.

Code:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Feature scaling is especially important for algorithms such as Logistic Regression because it prevents variables with larger ranges from dominating the model.

---

7. Split the Dataset into Training and Testing Sets

I separated the processed data into training and testing subsets using stratified sampling to preserve class distribution.

Code:

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

This ensured consistent evaluation and reliable testing performance.

8. Integrated the Preprocessing Pipeline with Model Training

The processed dataset was successfully connected to the machine learning models implemented by the modeling team.

The preprocessing pipeline supported the following models:

- Logistic Regression
- Random Forest
- XGBoost

This integration allowed the team to generate professional evaluation metrics and visualizations.

Results and Analysis

The preprocessing pipeline successfully transformed the raw OULAD dataset into a clean and machine-learning-ready format. After preprocessing, the dataset was used to train and evaluate three classification models.

The pipeline enabled the generation of the following professional visualizations and evaluation metrics:

Visualization / Metric	Purpose
Confusion Matrix	Evaluate classification performance
ROC Curve	Measure model discrimination ability
Precision-Recall Curve	Evaluate minority class prediction
Feature Importance	Identify influential features
Metrics Dashboard	Compare all models side-by-side

Key preprocessing outcomes include:

- Successfully encoded categorical variables into numerical representations.
- Eliminated missing values from the dataset.
- Standardized feature scales for improved model compatibility.
- Maintained balanced class distribution using stratified train-test splitting.

- Prepared the dataset for advanced ensemble models such as Random Forest and XGBoost.

The preprocessing pipeline provided a strong foundation for accurate model training and evaluation.

Key Findings

- Standardization improved model stability and ensured consistent feature scaling across algorithms.
- Label Encoding allowed categorical academic and behavioral data to be processed effectively by machine learning models.
- Stratified train-test splitting preserved dropout and non-dropout class balance, reducing sampling bias.
- The preprocessing workflow enabled smooth integration with professional visualizations and evaluation metrics.
- The prepared dataset supported high-performing ensemble models such as XGBoost and Random Forest.

Challenges and Solutions

Challenge	Solution
Some features contained missing values	Applied median imputation for numerical columns and mode imputation for categorical columns
Machine learning models could not process categorical text data	Used LabelEncoder to convert categorical values into numerical form
Features had different numerical scales	Applied StandardScaler to normalize feature ranges
Need for reproducible train-test results	Used `random_state=42` during dataset splitting
Risk of class imbalance during splitting	Used `stratify=y` to preserve class distribution

---

Conclusion

I successfully developed and implemented the data preprocessing pipeline for the student dropout prediction project. My work included feature selection, categorical encoding, missing value handling, feature scaling, and train-test splitting. These preprocessing steps ensured that the dataset was clean, balanced, and ready for machine learning model training.

As the **Pipeline Lead (Data & Preprocessing)**, I learned the importance of preparing high-quality data before model development. I also gained experience integrating preprocessing workflows with machine learning pipelines and professional visualizations. My preprocessing pipeline provided the foundation for the group's model evaluation, visualization generation, and future fine-tuning activities.